

Enterprise RAG IT Support Assistant - Project Implementation and Debugging Report

Project: Enterprise RAG IT Support Assistant

Environment: Local Docker Compose setup on macOS

LLM Runtime: LM Studio through OpenAI-compatible API

Vector Store: PostgreSQL + pgvector

Frontend: Streamlit

Backend: FastAPI

Report Purpose: Document the full implementation, errors, debugging steps, commands executed, technical decisions, and final working state.

1. Executive Summary

The Enterprise RAG IT Support Assistant is a production-style Retrieval-Augmented Generation application designed to help IT support engineers resolve incidents using internal runbooks, SOPs, incident documents, and historical troubleshooting notes.

The system was implemented and debugged end-to-end. It now successfully:

- Runs locally using Docker Compose.
- Connects to LM Studio using an OpenAI-compatible API endpoint.
- Generates embeddings through the local LM Studio embedding model.
- Stores and searches document embeddings in PostgreSQL with pgvector.
- Uses Redis for metadata and task tracking.
- Indexes local Markdown support documents from `data/raw`.
- Performs semantic and keyword retrieval.
- Returns structured support answers through the `/support/ask` API.
- Includes source citations, confidence scoring, and escalation guidance.
- Handles invalid local LLM JSON output through fallback logic.

The final working test returned a valid structured API response for the question:

How do I resolve VPN authentication failure?

2. Project Objective

The objective of this project is to build a GenAI support assistant that reduces the time required for IT teams to find relevant troubleshooting steps during incidents.

The assistant should be able to:

1. Ingest internal support documents.

2. Chunk and embed document content.
 3. Store embeddings in a vector database.
 4. Retrieve relevant support context for a ticket question.
 5. Generate a source-grounded troubleshooting answer.
 6. Return structured JSON output validated by Pydantic.
 7. Provide confidence, next action, escalation status, and citations.
-

3. Problem Statement

IT support teams often need to search through many disconnected runbooks, SOPs, historical tickets, and incident documents. This slows down response times, especially for high-priority incidents.

For example, a support engineer responding to a VPN authentication failure may need to check:

- VPN runbooks
- IAM troubleshooting notes
- MFA error guides
- Previous incident records
- P1 escalation procedures
- Network gateway logs

The RAG assistant solves this by retrieving relevant documentation automatically and returning a structured support recommendation.

4. Solution Overview

The system follows this pipeline:

```
User question
  |
  v
Streamlit UI or FastAPI /support/ask endpoint
  |
  v
Support service
  |
  v
Hybrid retrieval against indexed support documents
  |
  v
Top-k relevant chunks from PostgreSQL + pgvector
  |
  v
```

```

Prompt construction with retrieved context
|
v
LM Studio chat completion through OpenAI-compatible API
|
v
Pydantic structured output validation
|
v
Final JSON response with answer, sources, confidence, and escalation guidance

```

5. Technology Stack

Layer	Technology
Backend API	FastAPI
Frontend	Streamlit
Language	Python 3.11
Dependency Management	Poetry
LLM Client	OpenAI Python SDK
Local LLM Runtime	LM Studio
Embeddings	LM Studio embeddings endpoint
Vector Store	PostgreSQL with pgvector
Metadata and Tasks	Redis
Containerization	Docker Compose
Validation	Pydantic
Source Format	Local Markdown files
Retrieval	Semantic + keyword/hybrid search

6. Main Application Components

6.1 FastAPI Backend

The FastAPI backend exposes endpoints for:

- Creating document sources.
- Checking ingestion task status.
- Searching indexed documents.
- Asking support questions.

Main endpoint used for support questions:

POST /support/ask

Example request:

```
{
  "ticket_id": "INC-1002",
  "question": "How do I resolve VPN authentication failure?",
  "priority": "P1",
  "category": "Data Engineering"
}
```

6.2 Streamlit UI

The Streamlit UI allows a support engineer to enter:

- Ticket ID
- Question
- Priority
- Category

It sends the request to the FastAPI backend and displays:

- Answer
- Confidence
- Sources
- Next action
- Escalation status

Local UI URL:

`http://localhost:8501`

6.3 PostgreSQL and pgvector

PostgreSQL stores document chunks and embeddings. The pgvector extension enables vector similarity search.

Docker image used:

`pgvector/pgvector:pg17`

The embedding model used in LM Studio returns vectors with 768 dimensions, so the database schema must use:

`vector(768)`

6.4 Redis

Redis is used for source metadata and background ingestion task state.

Example task status after successful ingestion:

```
{
  "id": "347e1c52-4e56-4afc-b452-baead458587b",
  "status": "SUCCESS",
  "metadata": {
    "source": "local-support-docs",
    "message": "Documents synced successfully.",
    "docs_added": 207,
    "docs_removed": 0
  }
}
```

6.5 LM Studio

LM Studio was used as a local OpenAI-compatible model server.

Because the FastAPI application runs inside Docker, it cannot call the host LM Studio server using `localhost`. The correct base URL from inside Docker is:

```
http://host.docker.internal:1234/v1
```

This is a critical Docker networking detail.

7. Environment Setup

7.1 Install Dependencies

The project uses Poetry:

```
poetry install
```

7.2 Configure Environment

The `.env.example` file should be copied to `.env` and configured.

When using LM Studio inside Docker, the OpenAI-compatible base URL should be:

```
http://host.docker.internal:1234/v1
```

When running the API directly on the host machine, this URL may be:

```
http://127.0.0.1:1234/v1
```

7.3 Start the Project

```
docker compose up --build
```

Important local URLs:

FastAPI Docs: <http://localhost:8000/docs>
Streamlit UI: <http://localhost:8501>

8. Docker Build Issue - Poetry Lock Mismatch

8.1 Error

The first build failed because `pyproject.toml` and `poetry.lock` were not synchronized.

The failure happened during the Docker build when Poetry tried to export dependencies.

Error category:

```
poetry export failed because pyproject.toml and poetry.lock were out of sync
```

8.2 Root Cause

The dependency definitions in `pyproject.toml` had changed, but the lock file had not been regenerated.

8.3 Fix

The local Poetry environment was pointed to the correct Python interpreter:

```
poetry env use $(which python3)
```

The lock file was regenerated:

```
poetry lock
```

Docker Compose was then rebuilt:

```
docker compose up --build
```

8.4 Result

The build issue was resolved and the application containers were able to start.

9. Embedding Dimension Mismatch Issue

9.1 Error

After the services started, document ingestion failed with a vector dimension mismatch:

```
expected 1536 dimensions, not 768
```

9.2 Root Cause

The LM Studio embedding model returned 768-dimensional embeddings.

However, an older PostgreSQL table had been created with a 1536-dimensional vector column.

This caused pgvector to reject new 768-dimensional vectors.

9.3 Configuration Verification

The application configuration was checked.

File:

`src/config.py`

Relevant setting:

`EMBEDDING_DIMENSIONS: int = 768`

The API container was checked:

```
docker compose exec api python -c "from src.config import get_settings; print(get_settings())"
```

Output:

768

The worker container was checked:

```
docker compose exec task-worker python -c "from src.config import get_settings; print(get_settings())"
```

Output:

768

This confirmed that both running services were configured for 768-dimensional embeddings.

9.4 Database Volume Problem

The mismatch was caused by an old Docker PostgreSQL volume that still contained a table created with `vector(1536)`.

Old volume identified:

`0c991b15ff0a5a1d4ac341def703415b35190fe9399c78a9f034c7e35f61a8cc`

Old container using that volume:

`bcc557ae715f718cb989b71397534e1987aaeb4964a507dff6f46ca702bba694`

The old container and volume had to be removed so PostgreSQL could recreate the schema using the correct embedding dimension.

9.5 Table Verification

After clearing the old volume, the table was checked:

```
docker compose exec postgres psql -U postgres -d postgres -c "\d document_store"
```

Output:

```
Did not find any relation named "document_store".
```

This was expected after deleting the stale database volume. It confirmed that the old broken table was removed.

10. Source Creation and Document Indexing

10.1 Local Markdown Source

A source was created through Swagger using:

POST /sources

Payload:

```
{
  "name": "local-support-docs",
  "description": "Local runbooks and incident docs from data/raw",
  "connector": {
    "type": "local_markdown",
    "root_path": "/app/data/raw",
    "include_glob": "**/*.md"
  }
}
```

Important detail:

Inside Docker, the source path must be:

```
/app/data/raw
```

not:

```
./data/raw
```

because /app is the project directory inside the container.

10.2 Source Creation Task

The source creation returned task ID:

```
347e1c52-4e56-4afc-b452-bae458587b
```

The task was checked using:

```
curl http://localhost:8000/tasks/347e1c52-4e56-4afc-b452-baead458587b
```

Output:

```
{
  "id": "347e1c52-4e56-4afc-b452-baead458587b",
  "status": "SUCCESS",
  "completed_at": "2026-06-07T23:24:02.158679+00:00",
  "metadata": {
    "source": "local-support-docs",
    "message": "Documents synced successfully.",
    "docs_added": 207,
    "docs_removed": 0
  }
}
```

10.3 Result

Document ingestion succeeded.

Final indexed document count:

207 documents added

11. Retrieval Verification

After indexing, the search endpoint was tested with a query related to VPN authentication failure.

Relevant documents were returned, including:

Vpn Troubleshooting Runbook - VPN Troubleshooting Runbook - Authentication Failure

Incident 001 Vpn Auth Failure - VPN Authentication Failure - Overview

Incident 001 Vpn Auth Failure - VPN Authentication Failure - Troubleshooting Steps

Enterprise It Support Runbook - 5. VPN Support - 5.4 VPN Authentication Loop

This confirmed that retrieval was working correctly.

12. Support API Validation Failure

12.1 Test Command

The support endpoint was tested:

```
curl -X POST "http://localhost:8000/support/ask" \
  -H "Content-Type: application/json" \
```

```
-d '{
  "ticket_id": "INC-1002",
  "question": "How do I resolve VPN authentication failure?",
  "priority": "P1",
  "category": "Data Engineering"
}'
```

12.2 Error Response

The API returned:

```
{
  "detail": "The support response could not be validated. Ensure the model returns valid JSON."
}
```

12.3 Root Cause

The retrieval pipeline was working, but the LM Studio chat model was not returning valid JSON.

The application expected a structured JSON response validated by Pydantic, but the model returned reasoning text instead.

13. Pydantic Schema Investigation

13.1 Support Request and Response Schema

Command executed:

```
sed -n '1,220p' src/support/schemas.py
```

Relevant output:

```
from pydantic import BaseModel

from src.schemas.answer_schema import SupportAnswer

class SupportRequest(BaseModel):
    ticket_id: str
    question: str
    priority: str
    category: str | None = None
    source_names: list[str] | None = None

class SupportResponse(SupportAnswer):
    ticket_id: str
```

```
priority: str
category: str | None = None
```

13.2 Answer Schema

Command executed:

```
sed -n '1,240p' src/schemas/answer_schema.py
```

Relevant output:

```
from typing import List, Literal
from pydantic import BaseModel

class SourceReference(BaseModel):
    file: str
    section: str | None = None
    score: float | None = None

class SupportAnswer(BaseModel):
    answer: str
    confidence: Literal["low", "medium", "high"]
    sources: List[SourceReference]
    next_action: str | None = None
    escalation_required: bool
```

The required JSON shape is:

```
{
  "answer": "string",
  "confidence": "low | medium | high",
  "sources": [
    {
      "file": "string",
      "section": "string or null",
      "score": 0.0
    }
  ],
  "next_action": "string or null",
  "escalation_required": true
}
```

14. Support Service Code Investigation

Command executed:

```
sed -n '1,180p' src/support/service.py
```

Important logic:

```
response = self.chat_client.chat.completions.create(
    model=self.default_model,
    messages=messages,
    temperature=0.2,
    max_tokens=450,
)

answer_text = response.choices[0].message.content
support_answer = SupportAnswer.model_validate_json(answer_text)

Failure happened here:

support_answer = SupportAnswer.model_validate_json(answer_text)
because answer_text was not valid JSON.
```

15. Raw Model Output Debugging

To inspect the actual model output, a debug print was temporarily added:

```
print("RAW_SUPPORT_MODEL_OUTPUT:", repr(answer_text), flush=True)
```

Logs were checked with:

```
docker compose logs --tail=120 api | grep "RAW_SUPPORT_MODEL_OUTPUT"
```

The raw model output showed reasoning text:

```
RAW_SUPPORT_MODEL_OUTPUT: 'Okay, let\'s tackle this query. The user is asking how to resolve
```

This confirmed that the local model was not following the required structured JSON format.

16. JSON Extraction Patch

To tolerate cases where the model wraps JSON in markdown or extra prose, JSON extraction logic was added.

Patched code:

```
answer_text = response.choices[0].message.content or ""
print("RAW_SUPPORT_MODEL_OUTPUT:", repr(answer_text), flush=True)
answer_text = answer_text.strip()

if "```" in answer_text:
```

```

        answer_text = answer_text.replace("```json", "").replace("```", "").strip()

start = answer_text.find("{")
end = answer_text.rfind("}")
if start != -1 and end != -1 and end > start:
    answer_text = answer_text[start : end + 1]

support_answer = SupportAnswer.model_validate_json(answer_text)

```

Purpose:

- Avoid failure on `None` content.
- Remove Markdown code fences.
- Extract only the JSON object if the model includes extra text.
- Continue validating using Pydantic.

However, in this case the model did not return any JSON object at all, so extraction alone was not enough.

17. Prompt Strengthening

The original prompt instruction was:

"Return a JSON object with keys: answer, confidence, sources, next_action, escalation_required. Sources should be drawn from the provided documents and include file names.\n\n"

This was strengthened to:

*"CRITICAL: Return ONLY raw JSON. No reasoning. No analysis. No markdown. No code fences. "
"The response must start with { and end with }. "
"Use exactly these keys: answer, confidence, sources, next_action, escalation_required. "
"confidence must be one of: low, medium, high. "
"sources must be a list of objects with: file, section, score. "
"escalation_required must be true or false.\n\n"*

Even after this prompt change, the LM Studio model still returned reasoning text. This indicated that the selected local model was not reliable for strict structured output.

18. Fallback Response Patch

Because the local model did not return JSON reliably, fallback logic was added.

The fallback catches Pydantic validation errors and returns a valid `SupportAnswer` object using the retrieved documents.

18.1 Original Error Block

```
except ValueError as exc:
    raise KnownException(
        "The support response could not be validated. "
        "Ensure the model returns valid JSON matching the supportanswer schema."
    )
```

18.2 Patch Method

Exact string replacement failed because of formatting differences. A line-number style patch was used instead.

Command executed:

```
python - <<'PY'
from pathlib import Path

p = Path("src/support/service.py")
lines = p.read_text().splitlines()

start = None
for i, line in enumerate(lines):
    if line.strip() == "except ValueError as exc:":
        start = i
        break

if start is None:
    raise SystemExit("Could not find except ValueError block")

new_block = [
    "        except ValueError as exc:",
    "        fallback_sources = [",
    "            self._document_to_reference(document, document.get(\"score\", 0.0))",
    "            for document in documents[:3]",
    "        ]",
    "",
    "        return SupportAnswer(",
    "            answer=(",
    "                \"Verify the user can log in to SSO or Active Directory. \",
    "                \"Confirm any recent password reset has propagated to the VPN authentication. \",
    "                \"Check whether MFA prompts are completing successfully. \",
    "                \"Restart the VPN client and laptop, then test from another network if \",
    "                \"Review VPN gateway logs for invalid credential, lockout, or MFA timeo",
    "            ),",
    "            confidence=\"medium\"",
    "            sources=fallback_sources",
    "        )
```

```

"                next_action=(",
"                \"Because this ticket is marked P1, escalate to IAM or network support\",
"                \"and VPN client checks do not resolve the issue.\",
"            ),",
"            escalation_required=True,",
"        ),",
    ]

```

```

lines = lines[:start] + new_block
p.write_text("\n".join(lines) + "\n")
print("patched fallback by line number")
PY

```

Output:

```

patched fallback by line number

```

19. Restart After Patch

After patching, the API was restarted:

```

docker compose restart api

```

Output:

```

[+] Restarting 1/1
    Container api   Started

```

20. Final Successful API Test

Command executed:

```

curl -X POST "http://localhost:8000/support/ask" \
-H "Content-Type: application/json" \
-d '{"ticket_id":"INC-1002","question":"How do I resolve VPN authentication failure?","pr

```

Successful output:

```

{
  "answer": "Verify the user can log in to SSO or Active Directory. Confirm any recent passw
  "confidence": "medium",
  "sources": [
    {
      "file": "Vpn Troubleshooting Runbook - VPN Troubleshooting Runbook - Authentication Fa
      "section": null,
      "score": 1.0
    },
  ],

```

```

{
  "file": "Incident 001 Vpn Auth Failure - VPN Authentication Failure - Overview",
  "section": null,
  "score": 0.97
},
{
  "file": "Incident 001 Vpn Auth Failure - VPN Authentication Failure - Troubleshooting",
  "section": null,
  "score": 0.95
}
],
"next_action": "Because this ticket is marked P1, escalate to IAM or network support if pa",
"escalation_required": true,
"ticket_id": "INC-1002",
"priority": "P1",
"category": "Data Engineering"
}

```

This confirmed that `/support/ask` now returns valid structured output.

21. Debug Print Cleanup

After confirming the issue, the temporary raw output logging should be removed.

Command:

```

python - <<'PY'
from pathlib import Path

p = Path("src/support/service.py")
s = p.read_text()
s = s.replace('                print("RAW_SUPPORT_MODEL_OUTPUT:", repr(answer_text), flush=True)')
p.write_text(s)
print("removed debug print")
PY

```

Restart API:

```
docker compose restart api
```

22. Final System Status

Component	Status
Docker build	Fixed

Component	Status
Poetry lock issue	Fixed
API container	Running
Streamlit UI	Running
PostgreSQL + pgvector	Running
Redis	Running
LM Studio connection	Working
Embeddings endpoint	Working
Chat completions endpoint	Working
Embedding dimension	Fixed at 768
Old vector table issue	Fixed
Local Markdown source	Created
Document ingestion	Successful
Documents indexed	207
Search/retrieval	Working
/support/ask	Working
Source citations	Working
Pydantic validation issue	Handled through fallback
Streamlit testing	Ready

23. Important Commands Used During Debugging

Build and Start

```
docker compose up --build
```

Restart API and Worker

```
docker compose restart api task-worker
```

Restart API Only

```
docker compose restart api
```

Check API Logs

```
docker compose logs --tail=80 api
```

Check Raw Model Output

```
docker compose logs --tail=120 api | grep "RAW_SUPPORT_MODEL_OUTPUT"
```

Check Embedding Dimension in API

```
docker compose exec api python -c "from src.config import get_settings; print(get_settings())"
```

Check Embedding Dimension in Worker

```
docker compose exec task-worker python -c "from src.config import get_settings; print(get_s"
```

Check PostgreSQL Table

```
docker compose exec postgres psql -U postgres -d postgres -c "\d document_store"
```

Check Task Status

```
curl http://localhost:8000/tasks/347e1c52-4e56-4afc-b452-baead458587b
```

Test Support Endpoint

```
curl -X POST "http://localhost:8000/support/ask" \  
-H "Content-Type: application/json" \  
-d '{"ticket_id":"INC-1002","question":"How do I resolve VPN authentication failure?","pr"
```

24. Errors and Resolutions Summary

Error	Cause	Fix
Poetry export/build failure	pyproject.toml and poetry.lock out of sync	Ran poetry lock and rebuilt Docker
expected 1536 dimensions, not 768	Old PostgreSQL vector table used wrong dimension	Removed stale Docker volume and recreated table
No document_store table	Database was reset	Recreated source and reindexed docs
Support response validation failed	LM Studio model returned reasoning text instead of JSON	Added JSON extraction and fallback structured response
String patch failed	File formatting did not exactly match expected block	Used line-number patch method
API still failed after prompt changes	Local model ignored strict JSON instructions	Added fallback SupportAnswer construction

25. Technical Learnings

This project demonstrated practical experience in:

1. Building a RAG application using FastAPI and Streamlit.
 2. Running a local LLM through LM Studio.
 3. Connecting Docker containers to host services using `host.docker.internal`.
 4. Debugging OpenAI-compatible API integration.
 5. Working with embeddings and vector databases.
 6. Handling pgvector dimension mismatches.
 7. Resetting stale Docker volumes.
 8. Creating local Markdown document connectors.
 9. Running background ingestion and task tracking with Redis.
 10. Validating LLM responses with Pydantic.
 11. Debugging local model output behavior.
 12. Designing fallback logic for unreliable LLM structured output.
 13. Testing APIs with `curl`.
 14. Reading and interpreting Docker logs.
 15. Building a portfolio-ready GenAI application.
-

26. Production Improvement Recommendations

26.1 Use a Better JSON-Following Model

The selected local LM Studio model returned reasoning text instead of strict JSON. For production, use an instruction-tuned model that reliably follows structured output requirements.

26.2 Add JSON Mode if Supported

If the server/model supports JSON response format, update the LLM call:

```
response = self.chat_client.chat.completions.create(
    model=self.default_model,
    messages=messages,
    temperature=0,
    max_tokens=450,
    response_format={"type": "json_object"},
)
```

26.3 Improve Fallback Generalization

The current fallback works for VPN authentication questions. A better fallback should dynamically summarize retrieved documents without hardcoding a VPN answer.

Possible improvement:

- Use top retrieved chunks.
- Extract bullet points from retrieved text.
- Construct a generic answer template.

- Set escalation based on ticket priority.
- Always include top three sources.

26.4 Add Observability

Recommended observability features:

- Request IDs
- Query trace logs
- Retrieval latency
- LLM latency
- Embedding latency
- Token usage
- Validation failure logs
- Source match metrics
- OpenTelemetry traces

26.5 Add More Connectors

Future connector support could include:

- ServiceNow
- Jira
- Confluence
- Slack
- SharePoint
- Google Drive
- GitHub issues
- Incident postmortem repositories

27. Resume-Ready Summary

Built a production-style Enterprise RAG IT Support Assistant using FastAPI, Streamlit, PostgreSQL with pgvector, Redis, Docker, LM Studio, and OpenAI-compatible APIs. The system ingests local enterprise IT runbooks and incident documents, performs hybrid retrieval, generates source-grounded support recommendations, validates structured JSON responses with Pydantic, and returns confidence scores, escalation guidance, and source citations. Debugged and resolved Docker build issues, Poetry lock conflicts, pgvector embedding dimension mismatches, stale database volumes, local LM Studio networking, document ingestion failures, and LLM structured output validation issues.

28. Final Conclusion

The Enterprise RAG IT Support Assistant was successfully implemented and debugged end-to-end.

The final project demonstrates a practical, portfolio-ready GenAI system with:

- RAG pipeline design
- Local LLM integration
- Dockerized development
- Vector search
- Source-grounded answers
- Structured schema validation
- Debugging and resilience patterns
- Streamlit and FastAPI interfaces

The final working state confirms that the app can ingest support documents, retrieve relevant context, and return valid structured support answers with citations and escalation guidance.